EuroSAT Deep Learning Project

Detailed 3-Person Remote Team Work Plan

Proposed topic: Satellite Land-Use Classification Using CNN Optimization and Transfer Learning on EuroSAT

Purpose: organize the assignment so that all three team members work fairly and in parallel, while keeping experiments controlled and ensuring that everyone understands the complete project for the live defense.

1. Project Strategy

The project will be completed stage by stage. At each experimental stage, the three members receive different controlled experiments, run them in parallel, then meet to compare results and decide which configuration moves forward. This avoids giving one entire assignment task to one person and keeps the workload more balanced.

Core experimental rule: when testing one factor, keep the other important settings fixed. This makes the comparison meaningful because the team can identify what caused a change in performance.

2. Overall Workflow

- Stage 1 - Common dataset pipeline and baseline CNN
- Stage 2 - Parallel architecture experiments -> compare -> select best architecture
- Stage 3 - Parallel optimizer and learning-rate experiments -> compare -> select best training configuration
- Stage 4 - Parallel augmentation experiments -> compare -> select augmentation strategy
- Stage 5 - Train and evaluate the final optimized scratch CNN
- Stage 6 - Parallel transfer-learning experiments -> compare scratch vs. pretrained approaches
- Stage 7 - Final evaluation and analysis
- Stage 8 - Report and presentation
- Stage 9 - Checkpoints and final notebook verification
- Stage 10 - Individual defense preparation

3. Stage 1 - Dataset and Common Baseline

Goal

Create one shared starting point before the team separates into experiments. Everyone must use the same data split, preprocessing, baseline architecture, training code and evaluation code.

3.1 EuroSAT dataset preparation

Load EuroSAT using torchvision.datasets.EuroSAT. The dataset contains 10 predefined land-use classes: AnnualCrop, Forest, HerbaceousVegetation, Highway, Industrial, Pasture, PermanentCrop, Residential, River and SeaLake.

- Create one fixed training / validation / test split.
- Set a random seed so that all members receive the same split.

- Create train_loader, val_loader and test_loader.
- Use training data to update weights, validation data to compare experiments, and reserve the test data for final evaluation.

3.2 Baseline CNN

Build a simple multi-layer CNN from scratch, with no pretrained weights. A reasonable baseline can contain three convolutional blocks followed by flattening and fully connected layers, ending with 10 output neurons - one for each EuroSAT class.

The baseline is a reference model, not the final model. The team should avoid spending too much time optimizing it before the controlled experiments begin.

3.3 Shared training and evaluation functions

- Training loss for each epoch
- Validation loss for each epoch
- Validation accuracy for each epoch
- Per-class accuracy
- Learning-curve plotting
- Normalized confusion matrix
- Checkpoint saving and loading

End-of-stage deliverable: one working baseline model and shared reusable code. Save the baseline checkpoint as baseline.pth and record its results.

4. Stage 2 - Architecture Experiments

Goal: determine which scratch-CNN architecture is most suitable for EuroSAT. Everyone starts from the same baseline and changes only the assigned architectural factor.

Member	Main experiment	Example tests	Question answered
Person 1	Network depth	2 vs 3 vs 4 convolutional layers	Does increasing CNN depth improve performance?
Person 2	Kernel size	3x3 vs 5x5 vs 7x7	Which receptive-field configuration works best?
Person 3	BatchNorm / stride / padding	Baseline vs BatchNorm; controlled stride/padding changes	Do these changes improve training and generalization?

Meeting 1 - Architecture comparison

Each member brings the results of their experiments. Combine them into one benchmark table containing model change, parameter count, peak validation accuracy, runtime/latency and generalization gap. Discuss the results and select one architecture to move to Stage 3.

5. Stage 3 - Optimizer and Learning-Rate Experiments

Goal: stop changing the architecture and determine how the selected architecture should be trained. All members use the same architecture, split, epochs, batch size, loss and evaluation procedure.

Member	Optimizer	Learning rates	Number of runs
Person 1	SGD + Momentum	1e-2, 1e-3, 1e-4	3
Person 2	Adam	1e-2, 1e-3, 1e-4	3

Person 3	AdamW	1e-2, 1e-3, 1e-4	3

This produces nine controlled experiments with an equal workload. Record peak validation accuracy, validation loss, runtime and generalization gap for every run.

Learning-rate scheduler

After the most promising optimizer/learning-rate configuration is identified, run a smaller controlled comparison of that configuration with and without a learning-rate scheduler. There is no need to test every scheduler with every optimizer unless the team has extra time.

Meeting 2 - Hyperparameter comparison

Combine all nine results, compare them, and select the optimizer, initial learning rate and scheduler decision that will be used in the next stage.

6. Stage 4 - Data Augmentation Experiments

Goal: keep the selected architecture, optimizer and learning rate fixed and investigate whether augmentation improves generalization.

Member	Augmentation experiment	Purpose
Person 1	Random flips + random cropping	Test basic spatial augmentation
Person 2	Affine transformations + ColorJitter	Test geometric and appearance variation
Person 3	MixUp or CutMix	Test an advanced augmentation technique

Use the no-augmentation configuration as the control. Compare both validation performance and the generalization gap; an augmentation can be useful even if its training accuracy is lower.

Meeting 3 - Augmentation comparison

Compare the augmentation strategies and decide which pipeline will be used for the final scratch model. Discuss why each method helped or hurt rather than selecting a result only by its highest accuracy.

7. Stage 5 - Final Optimized Scratch CNN

Combine the best choices found in the previous stages: architecture, kernel/configuration, optimizer, learning rate, scheduler decision and augmentation strategy. Train this configuration as the team's final CNN built from scratch.

- Save the model as best_scratch_cnn.pth.
- Generate training and validation loss curves.
- Generate training and validation accuracy curves.
- Calculate per-class accuracy.
- Generate a normalized confusion matrix.
- Record final test accuracy, parameter count, runtime/latency and generalization gap.

8. Stage 6 - Transfer Learning

Goal: compare the optimized scratch CNN with transfer learning. Choose one modern pretrained backbone together; ResNet-18 is a straightforward option. Replace its final classifier so that it outputs the 10 EuroSAT classes.

Member	Experiment	What is trainable?	Purpose
Person 1	Frozen feature extractor	New classification head only	Measure how useful pretrained features are without adapting the backbone
Person 2	Partial fine-tuning	Upper pretrained blocks + classifier	Test whether adapting high-level features improves EuroSAT performance
Person 3	Full fine-tuning	Entire backbone + classifier	Test maximum adaptation using a reduced learning rate

Meeting 4 - Transfer-learning comparison

Compare the final scratch CNN, frozen pretrained model, partially fine-tuned model and fully fine-tuned model. Focus on final test metrics, convergence speed, data efficiency, trainable parameters and training time.

9. Stage 7 - Final Evaluation and Analysis

At this point, stop adding new experiments unless a required result is missing. Identify the best scratch model and best transfer-learning model, then prepare the final figures, tables and interpretation.

- Learning curves
- Normalized confusion matrices
- Per-class accuracy
- Final test metrics
- Parameter counts
- Runtime/latency
- Generalization gap
- Convergence behavior
- Explanation of why the observed differences are meaningful

10. Stage 8 - Report and Presentation

The assignment requires a 6-8 page IEEE-style technical report. Writing can be divided, but all three members should review the complete report so that terminology and conclusions remain consistent.

Member	Primary writing responsibility
Person 1	Dataset, preprocessing and baseline methodology
Person 2	Architecture, optimizer, learning-rate and augmentation experiments
Person 3	Transfer learning, fine-tuning and transfer-learning comparison
All three	Results discussion, conclusion, final tables, proofreading and presentation

Suggested report structure

- 1. Introduction
- 2. Dataset and Preprocessing
- 3. Methodology: baseline CNN, architecture experiments, hyperparameters, augmentation and transfer learning
- 4. Experimental Results
- 5. Discussion
- 6. Conclusion

The presentation should tell the experimental story: starting baseline -> controlled experiments -> selected configuration -> optimized scratch CNN -> transfer learning -> final comparison and conclusions.

11. Stage 9 - Checkpoints and Final Notebook

Keep the important trained models clearly named because the evaluators may ask the team to load saved weights and run inference on unseen mini-batches.

- baseline.pth
- best_scratch_cnn.pth
- resnet_frozen.pth
- resnet_partial_finetuned.pth
- resnet_full_finetuned.pth

Before submission, execute the final notebook(s) from start to finish and confirm that all required cells, plots and checkpoint-loading steps work without runtime errors.

12. Stage 10 - Individual Defense Preparation

After development is complete, ownership ends: every member must understand the entire project. The individual defense is heavily weighted, and the assignment states that students may be asked to modify code live.

Each member should be able to explain

- EuroSAT dataset, labels, splits and DataLoaders
- Transforms and augmentation
- CNN layers and tensor dimensions
- Forward pass, loss, backward pass and optimizer
- Training and validation loops
- Kernel size, depth, stride, padding and BatchNorm experiments
- Learning rate, optimizers and schedulers
- Transfer learning, pretrained weights, freezing and fine-tuning
- Evaluation metrics and confusion matrices
- Saving/loading checkpoints and running inference

Mock defense

Run remote mock-defense sessions. One member shares the screen while the other two ask them to change a kernel size, add/remove BatchNorm, change an optimizer or learning rate, modify an augmentation, freeze/unfreeze a ResNet block, load a checkpoint and perform inference.

13. Remote Collaboration Rules

- Use one shared GitHub repository/codebase rather than exchanging separate notebook copies.
- Keep the dataset out of GitHub; each member can download EuroSAT locally.
- Use one shared results sheet and record every experiment immediately.
- Do not change shared settings such as the data split, seed or preprocessing without informing the team.
- Use clear experiment and checkpoint names instead of 'final_final2'.

- Every comparison meeting must end with a decision about what configuration moves forward.
- All members should pull the latest shared code before beginning a new stage.

14. Shared Experiment Record

For every run, record the following information in the shared results sheet:

- Experiment ID and member
- Architecture and architectural change
- Kernel size / stride / padding / BatchNorm setting
- Optimizer and learning rate
- Scheduler
- Augmentation
- Epochs and batch size
- Total parameters and trainable parameters
- Peak validation accuracy
- Final training and validation accuracy
- Test accuracy when appropriate
- Runtime/latency
- Generalization gap
- Checkpoint filename
- Short notes / observations

15. Final Workload Summary

Stage	Person 1	Person 2	Person 3	Team decision
Baseline	Together	Together	Together	Establish reference
Architecture	Depth	Kernel sizes	BN / stride / padding	Choose best architecture
Hyperparameters	SGD x 3 LR	Adam x 3 LR	AdamW x 3 LR	Choose optimizer/LR
Augmentation	Flip + crop	Affine + ColorJitter	MixUp/CutMix	Choose augmentation
Final scratch CNN	Together	Together	Together	Evaluate final scratch model
Transfer learning	Frozen	Partial fine-tune	Full fine-tune	Compare transfer approaches
Final analysis	Together	Together	Together	Draw conclusions
Report	Dataset/method	Experiments	Transfer learning	Review together
Defense	Everything	Everything	Everything	Mock defenses

16. Alignment with the Assignment PDF

This plan is designed around the assignment requirements: a standard multi-class computer-vision dataset and DataLoaders; a CNN built from scratch; training/validation metrics and diagnostics; architectural and hyperparameter comparisons; data augmentation; transfer learning with freezing and fine-tuning; comparative evaluation; saved checkpoints; live notebook execution; and individual mastery of the full code.

Source: Course Project Specification - Neural Network; Deep Learning Architecture & Transfer Learning, Programming for AI (2026-2027).